Assignment 9: Priority, Round Robin, Multilevel Queues, and Multiprocessor Scheduling

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

October 29, 2026

Concepts That Click

Deadline: two weeks from the date of issue — there is no automatic late penalty; if you need more time, ask before the deadline and an extension will be granted (see the course policy in the syllabus).
Answer every question. Show all working for Numerical and Design questions: draw the Gantt chart first, then give TAT, WT, and RT per process — a bare final answer without the chart earns partial credit only.
Conventions (as in the lecture): times in ms; a *lower* priority number means *higher* priority; ties go to the earlier arrival; in RR a process arriving at the instant a quantum expires joins the ready queue *before* the preempted process; TAT = completion − arrival, WT = TAT − τ, RT = first dispatch − arrival.
Reading: Silberschatz et al., *Operating System Concepts*, Ch. 5 §5.3 (pp. 192–199) and §5.5 (p. 200 onward) [1]; Stallings, *Operating Systems: Internals and Design Principles*, Ch. 9 §9.2 and Ch. 10 §10.1 [2].

9.1 Conceptual

Q1. A classmate claims: "Priority scheduling and SJF are different algorithms; RR with a big enough quantum is still fairer than FCFS; and MLQ and MLFQ differ only in the number of queues." Respond precisely [1].

(a) Show that SJF is a special case of priority scheduling by naming the priority it uses. Then explain why both suffer the same starvation problem, and how aging bounds it: with priorities 0 (highest) to 127 and aging that lowers a waiting process's priority number by 1 every 20 ms, what is the longest any process can wait before reaching priority 0?

(b) State exactly what RR becomes when q is at least the longest CPU burst, and why. Then explain why RR's real guarantee is the bound $(n-1)q$ on the wait for the next quantum rather than a lower average WT.

(c) Contrast MLQ and MLFQ in terms of how a process's queue is chosen and whether it can change. List the five parameters that define an MLFQ.

Q2. A server has eight cores and per-CPU ready queues.

(a) Distinguish asymmetric multiprocessing from SMP, and say which one this server uses and why a common ready queue would need a lock.

(b) Define processor affinity, distinguish soft from hard affinity, and give a reason (beyond caches) why affinity matters more on a NUMA machine.

(c) Explain push and pull migration, and the tension between load balancing and affinity. When should the scheduler migrate a process?

(d) A core supports two hardware threads. Explain what a memory stall is and the two levels at which scheduling then takes place.

9.2 Numerical

Q3. Five processes arrive as follows:

Process	Arrival	Burst τ	Priority
`P1`	0	7	3
`P2`	2	4	1
`P3`	3	6	4
`P4`	5	3	2
`P5`	6	5	1

For each policy draw the Gantt chart (arrivals marked on the axis) and tabulate completion, TAT, WT, and RT per process, with averages:

(a) non-preemptive priority;

(b) preemptive priority (note that `P2` and `P5` share priority 1);

(c) RR with $q = 2$;

(d) RR with $q = 4$.

(e) Which policy gives the lowest average WT and which the lowest average RT? Count the context switches (changes of running process) for (c) and (d).

Q4. Overheads and multiprocessors.

(a) A context switch costs $s = 0.2$ ms. Compute the useful fraction $q/(q + s)$ for $q = 1, 4, 20$ ms. For your schedule in Q3(c), how much switching overhead is added in total?

(b) Eight interactive processes are ready under RR with $q = 15$ ms. Ignoring switch time, what is the longest a process can wait for its next quantum? What quantum would bound it by 70 ms?

(c) Six jobs with bursts 7, 6, 5, 4, 3, 3 ms are ready at $t = 0$ on two CPUs. Give the finish time when J1–J3 are statically assigned to CPU0 and J4–J6 to CPU1, and when a common queue hands the next job (in the listed order) to the first free CPU (lower-numbered CPU on ties). Show both two-CPU Gantt charts and compare with the lower bound.

Concepts That Click

9.3 Design

Q5. An MLFQ has three queues: Q_0 RR $q = 2$, Q_1 RR $q = 4$, Q_2 FCFS. New arrivals enter Q_0; a process that uses its full quantum is demoted; higher queues preempt lower ones; a preempted process returns to the tail of its *current* queue and receives a fresh quantum when next dispatched. The workload is `P1` (arrival 0, $\tau = 10$), `P2` (1, 2), `P3` (3, 5), `P4` (9, 1).

(a) Trace the schedule: give the Gantt chart, marking the queue of each slice, and the contents of each queue at every decision point.

(b) Give completion, TAT, WT, and RT per process and the averages.

(c) Repeat with plain RR, $q = 2$, and compare the treatment of `P3` and `P4`. Explain which feature of MLFQ produced the difference.

(d) `P2`'s burst equals the Q_0 quantum. Is `P2` demoted? Justify from the rule.

Q6. Design a scheduler for a lab server that runs (i) kernel daemons, (ii) students' interactive shells and editors, and (iii) overnight batch simulations [1].

(a) Propose an MLQ: the queues, the algorithm in each, and the inter-queue policy. Explain how you prevent batch starvation, giving numbers for a time-slice split.

(b) Now students sometimes run long compilations from their shells. Explain why your MLQ handles this badly and redesign it as an MLFQ, specifying all five parameters.

(c) Explain how a student could game your MLFQ with tiny I/O requests, and the specific rule that defends against it.

References

[1] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. John Wiley and Sons, 2018.

[2] William Stallings. *Operating Systems: Internals and Design Principles*. Prentice Hall, 2001.

Concepts That Click